

 How to Add This to Your Resume

PROJECTS SECTION — Short Version (1–2 lines)

TelecomPlatform — Enterprise Telecom Backend (Cox-like ISP System) Built a production-grade, event-driven microservices platform using Spring Boot 3.2, Kafka, Redis, JWT/OAuth2, ELK Stack, and New Relic APM, simulating real-world ISP back-office operations including billing, usage tracking, and customer support.

PROJECTS SECTION — Standard Version (3–5 bullet points)

TelecomPlatform — Enterprise Telecom Backend Microservices (*Personal Project*) *Spring Boot 3.2 · Spring Cloud · Kafka · Redis · PostgreSQL · JWT · OAuth2 · ELK · New Relic*

- Designed and built 8 production-grade microservices (Auth, Customer, Billing, Plan, Usage, Support, Notification, API Gateway) modelled after real ISP back-office systems like Cox and Comcast
 - Implemented JWT authentication with refresh token rotation, Redis-based token blacklisting, account lockout after 5 failed attempts, and OAuth2 social login (Google, GitHub) using Spring Security 6
 - Built event-driven communication using Apache Kafka (7 topics, manual-ACK consumers, dead-letter queues) to decouple billing, notification, and usage services — achieving zero data loss on payment processing
 - Integrated Redis for multi-layer caching (10-min customer cache, 1-hr plan cache), rate limiting (100 req/s via Spring Cloud Gateway), and real-time usage counters for live dashboards
 - Configured ELK Stack with structured JSON logging (Logstash TCP appender) and New Relic APM for distributed tracing, JVM metrics, and alerting across all 8 services
 - Applied database-per-service pattern with PostgreSQL, Flyway migrations, table partitioning (usage data by month), and JPA batch inserts (size 50) for high-throughput usage ingestion
 - Implemented circuit breaker (Resilience4j), retry with exponential backoff, and graceful fallback responses in the API Gateway to ensure fault-tolerant inter-service communication
-

PROJECTS SECTION — Detailed Version (for senior/backend roles)

TelecomPlatform — Cox-Like ISP Backend Platform (*Personal Project* | 2024) GitHub: github.com/yourusername/telecom-platform

Tech Stack: Java 17 · Spring Boot 3.2 · Spring Cloud 2023 · Spring Security 6 · Hibernate/JPA · PostgreSQL 16 · Apache Kafka · Redis 7 · Docker · ELK Stack · New Relic · MapStruct · Flyway · JUnit 5 · Mockito

Architecture & Design:

- Designed a microservices architecture with 8 independently deployable services communicating via REST (synchronous) and Kafka (asynchronous), following database-per-service and event-driven patterns
- Implemented API Gateway using Spring Cloud Gateway with JWT validation, Redis rate limiting, Resilience4j circuit breakers, and service discovery via Netflix Eureka

Security:

- Built complete JWT authentication flow — access tokens (15 min), refresh tokens (7 days) with rotation, Redis blacklist on logout, and account lockout after 5 failed attempts
- Integrated Spring Security OAuth2 for Google and GitHub social login with automatic account provisioning
- Applied method-level security (@PreAuthorize) with role hierarchy: ADMIN > CSR > BILLING > CUSTOMER

Data & Messaging:

- Designed 7 Kafka topics (customer, billing, usage, support, notification, service-account, audit events) with manual acknowledgement, 3-retry policy, and dead-letter queue pattern
- Implemented Usage Service consuming 500+ usage events/second with PostgreSQL table partitioning by month and Hibernate batch inserts for high-throughput ingestion
- Configured multi-layer Redis caching with per-cache TTLs (customer: 10 min, plans: 1 hr, invoices: 15 min) using Spring Cache abstraction

Billing Engine:

- Built a complete billing lifecycle: invoice generation with tax calculation (8%), regulatory fee (\$5.99), proration support, automated overdue detection (daily cron), late fee application (\$15), and Stripe-like payment processing

Observability:

- Configured structured JSON logging (Logback + Logstash encoder) shipped to Elasticsearch via TCP, with Kibana dashboards for error rates, consumer lag, and service health
- Integrated New Relic Java Agent (via Docker multi-stage build) for transaction tracing, DB query performance, and Kafka throughput monitoring
- Exposed Prometheus metrics via Micrometer (HTTP latency, cache hit rate, JVM heap, Kafka lag) for Grafana dashboards

Testing:

- Achieved 80%+ test coverage on service layer using JUnit 5 + Mockito with 25+ unit tests across Auth, Customer, Billing, Plan, Usage, and Support services

- Used MockMvc for controller integration tests validating HTTP status codes, request validation, and error response format
-

SKILLS SECTION — What to Add/Strengthen

From this project, you can confidently list:

Backend:

- Spring Boot, Spring Cloud (Gateway, Eureka, OpenFeign, Config)
- Spring Security (JWT, OAuth2, Method Security)
- Spring Data JPA / Hibernate (Auditing, Batch, Partitioning)
- RESTful API Design, Pagination, Validation

Messaging & Caching:

- Apache Kafka (Producer, Consumer, Dead Letter Queue, Manual ACK)
- Redis (Caching, Pub/Sub, Rate Limiting, Token Blacklist)

Database:

- PostgreSQL (Multi-database, Partitioning, Indexing)
- Flyway (Schema migrations, Seed data)

DevOps & Observability:

- Docker, Docker Compose (Multi-service orchestration)
- ELK Stack (Elasticsearch, Logstash, Kibana)
- New Relic APM (Distributed Tracing, Alerting)
- Prometheus + Grafana (Metrics)

Architecture Patterns:

- Microservices, Event-Driven Architecture
- Circuit Breaker, Retry, Rate Limiting
- Database-per-Service, CQRS-lite
- JWT + OAuth2 Security

Testing:

- JUnit 5, Mockito, MockMvc
 - Unit Testing, Integration Testing
-

ONE-LINER SUMMARIES (pick one for your headline/objective)

"Built a Cox-like ISP platform with 8 Spring Boot microservices, Kafka, Redis, JWT/OAuth2, ELK Stack, and New Relic — handling billing, usage ingestion, customer management, and support ticketing."

"Designed and implemented an enterprise-grade telecom backend using Spring Boot microservices, event-driven Kafka architecture, Redis caching, and full observability stack (ELK + New Relic)."

"Developed a production-style ISP backend platform with microservices, JWT authentication, Apache Kafka event streaming, Redis caching, circuit breakers, and structured logging to ELK — covering the full SDLC from design to testing."

Interview Q&A — TelecomPlatform Project

120+ questions with detailed answers, organized by topic area. Covers: Architecture · Security · Kafka · Redis · Spring · Database · Testing · Observability · Behavioral

SECTION 1 — Architecture & Microservices Design

Q1. Walk me through the overall architecture of your telecom platform.

A: The platform follows a microservices architecture with 8 independently deployable services. All client traffic enters through an API Gateway (Spring Cloud Gateway on port 8080) which handles JWT validation, rate limiting, and routing to downstream services via Netflix Eureka for service discovery. Communication between services is either synchronous REST (via Feign clients) or asynchronous event-driven via Apache Kafka. Each service owns its own PostgreSQL database (database-per-service pattern), uses Redis for caching, and ships structured logs to an ELK Stack. New Relic APM instruments every service for distributed tracing and performance monitoring.

Q2. Why did you choose microservices over a monolith for this project?

A: The domain naturally decomposes into bounded contexts — billing concerns are separate from customer management, which is separate from usage tracking. With microservices:

- Each service can be scaled independently (e.g., Usage Service needs higher throughput than Plan Service)
- Fault isolation — a billing service failure doesn't bring down the plan catalogue
- Independent deployment — billing logic can be released without touching auth
- Different services could use different tech stacks if needed (e.g., Usage Service with a time-series DB)

That said, I was also aware of the trade-offs: increased operational complexity, distributed transaction challenges, and network latency. For a real production system, I'd start with a well-structured modular monolith and extract services as boundaries become clearer.

Q3. How do services discover each other? Why not hardcode URLs?

A: Services register themselves with Netflix Eureka on startup and send heartbeats every 30 seconds. When Service A needs to call Service B, it asks Eureka for B's current IP:port. The Spring Cloud Gateway routes to `lb://service-name` — the `lb://` prefix triggers client-side load balancing via Spring Cloud LoadBalancer, which queries Eureka. Hardcoding URLs would break as soon as a service moves to a different host or port, especially in containerized/Kubernetes environments where IPs are dynamic.

Q4. How does the API Gateway validate JWT tokens without calling the Auth Service every time?

A: The gateway validates the JWT locally using the shared HMAC-SHA256 secret. It parses the token using jjwt, verifies the signature and expiry, and extracts claims — all in memory, without a network call. This means:

- Zero latency overhead from token validation
- Auth Service doesn't become a bottleneck
- The trade-off is that if a token is compromised, it remains valid until expiry (15 minutes) — we mitigate this with the Redis blacklist on logout

After validation, the gateway injects X-User-Id, X-User-Email, and X-User-Roles as headers, which downstream services read to build their SecurityContext.

Q5. What is the database-per-service pattern and how did you implement it?

A: Each microservice has its own dedicated PostgreSQL database that only it can access. No other service queries another service's database directly — they must go through the service's REST API or consume Kafka events. I implemented this with separate PostgreSQL containers in Docker Compose (ports 5432–5437) and separate Flyway migration paths per service. The benefit is loose coupling — the Billing Service can change its schema without affecting the Customer Service. The downside is that JOINS across services aren't possible, so you either denormalize data (e.g., storing planName in ServiceAccount alongside the planId) or use API composition.

Q6. How do you handle distributed transactions across services?

A: I followed the Saga pattern (choreography-based) rather than 2-phase commit. When a customer adds a service account:

1. Customer Service creates the account and publishes SERVICE_ACCOUNT_ADDED to Kafka
2. Billing Service consumes this event and sets up billing
3. Notification Service sends an activation email

If Billing Service fails to process the event, Kafka retries 3 times. After that it goes to the Dead Letter Topic for manual review. There's no strict ACID atomicity across services — instead, we achieve eventual consistency. For critical operations like payments, I use idempotent producers and store transactionId with a UNIQUE constraint so duplicate payments are rejected.

Q7. What is the Circuit Breaker pattern and where did you use it?

A: A circuit breaker monitors calls to a downstream service. If the failure rate exceeds a threshold (50% in my config, measured over a sliding window of 10 calls), the circuit "opens" and subsequent calls immediately return a fallback response without attempting the network call. After 30 seconds in the open state, it allows a few test calls (half-open) — if they succeed, it closes again.

I used Resilience4j in the API Gateway for each downstream service:

- Circuit opens after 50% failure rate over 10 calls

- 30-second wait before retry
- Fallback returns a 503 Service Unavailable JSON response explaining the service is temporarily down

This prevents cascade failures — a slow Support Service won't cause the entire gateway to queue up threads and crash.

Q8. What is the difference between synchronous and asynchronous communication in your project?

A:

- **Synchronous (REST/Feign):** Used when the caller needs an immediate response — e.g., Plan Service returning plan details, Customer Service looking up a customer. The caller blocks until the response arrives.
- **Asynchronous (Kafka):** Used for side-effects that don't need to block the main request — e.g., after creating a customer, we publish a CUSTOMER_CREATED event. The Notification Service picks it up and sends a welcome email. The customer creation API responds immediately without waiting for the email to be sent.

Rule of thumb: use async when the consumer doesn't need to be in the request's critical path.

SECTION 2 — Security (JWT, OAuth2, Spring Security)

Q9. Explain your complete JWT authentication flow.

A:

1. User POSTs credentials to `/api/v1/auth/login`
2. `AuthenticationManager.authenticate()` verifies the BCrypt-encoded password (cost factor 12)
3. On success, we generate two tokens:
 - Access token: 15 minutes, contains `userId`, email, roles as claims, signed with HS256
 - Refresh token: 7 days, stored in PostgreSQL, contains only `userId` and a unique `jti`
4. Both tokens returned to client
5. Client sends access token in `Authorization: Bearer <token>` header
6. API Gateway validates signature, expiry, and Redis blacklist on every request
7. On expiry, client sends refresh token to `/auth/refresh-token` → old token revoked, new pair issued (token rotation)
8. On logout, access token is blacklisted in Redis with TTL equal to remaining validity

Q10. Why use both access tokens AND refresh tokens? Why not just one long-lived token?

A: Security defence-in-depth:

- Short-lived access tokens (15 min) limit the damage window if a token is stolen — it becomes useless quickly
 - Refresh tokens allow the user to stay logged in without re-entering credentials, but they're only sent to `/auth/refresh-token`, reducing their exposure
 - Token rotation (new refresh token on each use) means a stolen refresh token can only be used once before the legitimate user invalidates it
 - A single long-lived token would be vulnerable for its entire duration if intercepted
-

Q11. How does the token blacklist work? What if Redis goes down?

A: On logout, the access token is stored in Redis as `blacklist:token:<token>` with a TTL equal to its remaining validity. Every request through the gateway checks `hasKey()` on this prefix. If the token is present, the request is rejected.

If Redis goes down: the gateway would fail to check the blacklist. I handle this in two ways — the `JwtAuthenticationFilter` wraps the Redis check in a try-catch, so if Redis is unavailable, validation falls through to just signature+expiry checking (fail-open for availability). In a high-security environment, you'd fail-closed instead. The 15-minute token TTL is the safety net — even without blacklisting, stolen tokens expire quickly.

Q12. What is BCrypt and why cost factor 12?

A: BCrypt is a password hashing algorithm designed to be slow and adaptive. It applies a salt (preventing rainbow table attacks) and iterates a key derivation function 2^{cost} times. Cost factor 12 means 4096 iterations — takes ~300ms on a modern CPU. This makes brute-force attacks impractical even with GPU acceleration.

Cost 10 (Spring's default) is too fast by today's standards. Cost 14+ becomes noticeable to users (>1 sec login). Cost 12 is the current recommended balance.

Q13. Explain how Spring Security's method-level security works in your project.

A: `@EnableMethodSecurity(prePostEnabled = true)` activates Spring Security's AOP-based method interceptors. On each service, Spring creates a proxy around beans annotated with `@PreAuthorize`. When the method is called, the SpEL expression is evaluated against the current `SecurityContext`.

Example: `@PreAuthorize("hasAnyRole('ADMIN', 'CSR')")` on `createCustomer()` — if the authenticated user doesn't have ADMIN or CSR role, a `AccessDeniedException` is thrown before the method body executes. The `GlobalExceptionHandler` catches this and returns a 403 response.

Q14. How does OAuth2 login work in your platform? What happens when a user logs in with Google?

A:

1. Client redirects to `/api/v1/auth/oauth2/authorization/google`
2. Spring Security redirects to Google's OAuth2 authorization endpoint with `client_id`, `scope=email,profile`, and a `state` parameter (CSRF protection)
3. User approves → Google redirects back to our callback URL with an `authorization_code`
4. Spring Security exchanges the code for an access token at Google's token endpoint (server-to-server)
5. Spring fetches user info (email, name, profile picture) from Google's userinfo endpoint
6. Our OAuth2UserService checks if a user with that email exists:
 - If not: creates a new user with `oauth_provider=google`, `oauth_provider_id=<google_sub>`
 - If yes: links the provider to the existing account
7. We issue our own JWT tokens and redirect the client with them

This way, our JWTs are always our own tokens — we don't rely on Google's tokens after the initial auth.

Q15. What is the difference between authentication and authorisation? How does your project handle both?

A:

- Authentication = verifying *who* you are (login with email/password or OAuth2 → issues JWT)
- Authorisation = verifying *what* you're allowed to do (JWT contains roles → `@PreAuthorize` checks them)

In my project:

- Authentication is handled by the Auth Service — validates credentials and issues JWT
 - Authorisation happens at two layers:
 1. API Gateway level — validates token exists and isn't expired/blacklisted
 2. Service level — `@PreAuthorize` checks specific roles on individual endpoints (e.g., only `ROLE_ADMIN` can deactivate plans, only `ROLE_CSR` or `ROLE_ADMIN` can suspend customers)
-

Q16. Why Kafka instead of RabbitMQ or HTTP callbacks for async communication?

A: Key reasons for Kafka:

- **Message retention:** Kafka retains messages for 7 days (configurable) even after consumers read them. RabbitMQ deletes messages after acknowledgement. This means new services can replay historical events (e.g., a new audit service could replay all billing events from the past week)
- **Consumer groups:** Multiple services can independently consume the same topic at their own pace without interfering with each other
- **Throughput:** Kafka handles millions of messages/second — essential for Usage Service ingesting real-time network data
- **Ordering guarantees:** Messages in the same partition are strictly ordered — all usage events for an account go to the same partition (keyed by accountId) ensuring sequential processing
- **Pull model:** Consumers pull at their own rate, preventing overwhelming slow consumers

RabbitMQ is better for low-latency, complex routing scenarios with fewer consumers.

Q17. What is a consumer group and why does it matter?

A: A consumer group is a set of consumers that collectively consume a topic. Kafka distributes partitions among group members — each partition is consumed by exactly one member of the group at any time.

In my project:

- `telecom.billing.events` has 3 partitions
- `notification-billing-group` has 1 consumer in Notification Service → it reads all 3 partitions alone
- If I scale Notification Service to 3 replicas, each replica gets 1 partition — parallel processing

Different services use different group IDs: `notification-billing-group` and (hypothetically) `analytics-billing-group` would both receive ALL billing events independently. This is Kafka's publish-subscribe at scale.

Q18. What is manual acknowledgement in Kafka and why did you use it?

A: By default, Spring Kafka auto-commits the offset after the consumer method returns (even if processing threw an exception). With `ack-mode: manual`, the offset is only committed when `ack.acknowledge()` is explicitly called in the consumer code.

I used this in the Usage Service to ensure at-least-once delivery:

```
usageRepository.save(record);    // persist first  
ack.acknowledge();              // only then commit offset
```

If the save throws an exception, the offset is NOT committed. Kafka will redeliver the message to the same consumer (or another in the group) on restart. Without manual ACK, if the app crashes after offset commit but before the DB write, that event is permanently lost.

Q19. What is a Dead Letter Topic (DLT) and how does it work?

A: After a consumer fails to process a message N times (retries exhausted), instead of dropping it, Kafka forwards it to a Dead Letter Topic — typically named `<original-topic>.DLT`. This prevents one bad message from blocking the entire consumer (a "poison pill" scenario).

In my project:

- 3 retry attempts with exponential backoff
 - On 3rd failure → message published to e.g., `telecom.billing.events.DLT`
 - Operations team monitors DLT via Kafka UI (`localhost:9093`)
 - After fixing the root cause (e.g., a schema mismatch), messages can be replayed from the DLT
-

Q20. How do you ensure exactly-once semantics in Kafka producers?

A: Strictly speaking, Kafka offers:

- At-most-once: message might be lost (no retry)
- At-least-once: message might be duplicated (retry on failure) — my default
- Exactly-once: requires idempotent producers + transactional API

For my billing producer, I used:

- `enable.idempotence=true` — Kafka deduplicates retried messages using producer ID + sequence number
- `acks=all` — leader waits for all in-sync replicas to acknowledge
- `max.in.flight.requests.per.connection=5` — allows pipelining while maintaining order

For the Payment entity, I store `transactionId` with a UNIQUE constraint — even if a duplicate payment event reaches Billing Service, the DB insert would fail with a unique violation, which I catch and treat as idempotent.

Q21. How does partitioning affect message ordering in your Usage Service?

A: Within a single partition, Kafka guarantees strict ordering. Across partitions, there is no ordering guarantee. In my Usage Service, I key usage events by `accountId`:

```
kafkaTemplate.send("telecom.usage.events", accountId.toString(), usagePayload);
```

Kafka hashes the key to determine the partition. All events for the same account go to the same partition → they're processed in order. This matters for usage aggregation — if events for the same account went to different partitions and were processed out of order, you could get incorrect usage totals.

SECTION 4 — Redis

Q22. How did you use Redis in this project? List all the use cases.

A: Four distinct use cases:

1. **JWT Token Blacklist (Auth Service):** Stores invalidated access tokens as `blacklist:token:<jwt>` with TTL = remaining token lifetime. $O(1)$ lookup on every request.
2. **Application-level Cache (Customer, Plan, Billing):** Spring Cache abstraction (`@Cacheable`, `@CacheEvict`) backed by Redis. Customer cache: 10 min TTL; Plan cache: 1 hour; Invoice cache: 15 min.
3. **Rate Limiting (API Gateway):** Spring Cloud Gateway's `RequestRateLimiter` filter uses Redis to implement the Token Bucket algorithm — 100 requests/second per IP, bursting to 200.
4. **Real-time Usage Counters (Usage Service):** After saving a usage record to PostgreSQL, we increment a Redis counter `usage:realtime:<accountId>:<usageType>` for real-time dashboard display without hitting the DB.

Q23. What caching strategy did you implement? What is cache-aside?

A: I used the cache-aside (lazy loading) pattern, which Spring's `@Cacheable` implements:

1. On a cache miss, the real method executes (DB query)
2. Result is stored in Redis
3. On subsequent calls, Redis returns the cached value

```
@Cacheable(value = "customers", key = "#customerId")  
  
public CustomerResponse getCustomerById(UUID customerId) {  
    return repo.findById(customerId)...;  
}
```

For updates, I use `@CacheEvict` to remove the stale entry:

```
@CacheEvict(value = "customers", key = "#customerId")  
  
public CustomerResponse updateCustomer(UUID customerId, UpdateRequest req) {...}
```

Alternative strategies I considered:

- Write-through: Update cache on every write — more complex, useful for write-heavy scenarios
 - Write-behind: Async DB write after cache update — risk of data loss
 - Cache-aside is simplest and suits read-heavy customer lookups.
-

Q24. What TTL (Time-To-Live) did you set and how did you decide?

A:

- Customer cache: 10 minutes — customers are read frequently but addresses/plans change occasionally. 10 min means stale data for at most 10 minutes, acceptable for non-financial data.
 - Plan cache: 1 hour — plans almost never change mid-day. 1-hour TTL dramatically reduces DB load with negligible staleness risk. Admin plan changes trigger `@CacheEvict(allEntries=true)`.
 - Invoice cache: 15 minutes — invoices are immutable once generated; only status changes on payment. 15 min is conservative.
 - Token blacklist: TTL = exact remaining lifetime of the JWT. This ensures the key auto-expires exactly when the token would expire anyway, preventing Redis memory bloat.
-

Q25. What is the Token Bucket algorithm and how does Redis implement rate limiting?

A: Token Bucket works like this: a bucket holds up to `burstCapacity` tokens. Tokens are added at `replenishRate` per second. Each request consumes 1 token. If the bucket is empty, the request is rejected (429 Too Many Requests).

Spring Cloud Gateway's `RequestRateLimiter` filter uses two Redis keys per client:

- `{prefix}.{key}.tokens` — current token count
- `{prefix}.{key}.timestamp` — last refill time

On each request, a Lua script atomically:

1. Calculates how many tokens to add since last refill
2. Adds tokens (up to burst capacity)
3. Checks if enough tokens exist for this request
4. Decrements and updates

Using a Lua script ensures atomicity — the check-and-decrement is one Redis operation, avoiding race conditions.

Q26. What is Spring Boot auto-configuration and how does it help?

A: Spring Boot's auto-configuration scans the classpath and conditionally creates beans. For example:

- If postgresql driver is on classpath → auto-configures a DataSource
- If spring-kafka is present → auto-configures KafkaTemplate
- If spring-data-redis is present → auto-configures RedisConnectionFactory

It uses `@ConditionalOnClass`, `@ConditionalOnMissingBean`, etc. You can override any auto-configured bean by defining your own `@Bean`. I overrode `RedisTemplate` to use `StringRedisSerializer` instead of the default Java serializer (for human-readable Redis keys) and defined `RedisCacheManager` for custom per-cache TTLs.

Q27. What is the N+1 query problem and how did you avoid it?

A: N+1 occurs with lazy-loaded associations. Example: loading 10 customers and then accessing each one's addresses would fire 1 query for customers + 10 queries for addresses = 11 queries. With `FetchType.LAZY` (Hibernate's default), associations are loaded on first access.

I avoided N+1 in several ways:

- Used `FetchType.EAGER` only for `User.roles` (small, always needed)
 - For `Invoice.lineItems` and `SupportTicket.comments`: used `FetchType.LAZY` and accessed them only within the same transaction. When the controller calls the service method, the transaction is still open (thanks to `@Transactional`).
 - For batch scenarios, would use `@EntityGraph` or `JOIN FETCH` in JPQL queries to eagerly load associations in a single query.
-

Q28. How does `@Transactional` work? What is a transaction propagation?

A: `@Transactional` wraps the method in a database transaction. Spring creates a proxy — when the method is called, the proxy begins a transaction, executes the method, and commits on success or rolls back on exception.

Propagation types I used:

- **REQUIRED (default):** join existing transaction or create new one. Most service methods use this.
- **REQUIRES_NEW:** suspend current transaction, create new one. Useful for audit logging that must persist even if main transaction rolls back.
- **SUPPORTS:** use transaction if one exists, else run without. Used for read-only queries called from both transactional and non-transactional contexts.

@Transactional(readOnly = true) is an optimization hint — Hibernate skips dirty checking, Hibernate flushes are disabled, and some databases use read replicas.

Q29. What is Flyway and why use it instead of ddl-auto: create?

A: Flyway is a database migration tool that applies versioned SQL scripts in order. Why not ddl-auto: create or update?

- create drops and recreates tables on every startup — data loss in production
- update tries to alter tables but misses things like dropping columns, changing constraints, or data migrations
- Flyway gives you full control: each migration is a numbered SQL file (V1__init.sql, V2__add_index.sql), applied once and tracked in a flyway_schema_history table
- Migrations are version-controlled, reviewed in PRs, and auditable
- Team members get the same schema automatically

I used baseline-on-migrate: true to handle existing databases gracefully.

Q30. What is JPA auditing and how did you use it?

A: JPA auditing auto-populates createdAt and updatedAt fields without manual code. I enabled it with @EnableJpaAuditing on the Application class and used:

```
@CreateDate @Column(name = "created_at", updatable = false)
```

```
private LocalDateTime createdAt;
```

```
@LastModifiedDate @Column(name = "updated_at")
```

```
private LocalDateTime updatedAt;
```

The entity must be annotated with @EntityListeners(AuditingEntityListener.class). Spring JPA intercepts @PrePersist and @PreUpdate hooks to set these fields automatically.

For user-based auditing (createdBy, modifiedBy), you'd implement AuditorAware<String> to return the current user from SecurityContextHolder.

Q31. How does table partitioning work in your Usage Service?

A: PostgreSQL table partitioning splits one logical table into multiple physical tables (child tables) based on a column value. I used PARTITION BY RANGE (occurred_at) — monthly partitions:

```
CREATE TABLE usage_records_2024_01 PARTITION OF usage_records
```

```
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
```

Benefits:

- Query performance: WHERE occurred_at BETWEEN '2024-01-01' AND '2024-01-31' only scans the January partition, not the entire table
- Data management: Dropping old data is instant (DROP TABLE usage_records_2023_01) vs. DELETE scanning the whole table
- Index size: Each partition's index is smaller = faster B-tree traversal

For the Usage Service handling millions of records, this is critical for query performance without adding a time-series database.

SECTION 6 — API Design & REST

Q32. How did you design your REST APIs? What conventions did you follow?

A: I followed REST conventions strictly:

- Nouns, not verbs in URLs: /api/v1/customers, not /api/v1/getCustomers
- HTTP methods for operations: GET (read), POST (create), PUT (full update), PATCH (partial update), DELETE (remove)
- Nested resources for relationships: /customers/{id}/accounts, /tickets/{id}/comments
- Consistent response envelope: ApiResponse<T> with success, message, data, errorCode, timestamp
- Versioning: /api/v1/ prefix — allows introducing /api/v2/ without breaking existing clients
- Pagination: Page<T> with Pageable for list endpoints
- HTTP status codes: 200 (success), 201 (created), 400 (validation error), 401 (unauthenticated), 403 (forbidden), 404 (not found), 503 (service unavailable)

Q33. What is the difference between @RequestBody, @PathVariable, and @RequestParam?

A:

- @PathVariable: extracts from URL path — GET /customers/{id} → @PathVariable UUID id
- @RequestParam: extracts from query string — GET /customers/search?status=ACTIVE → @RequestParam String status
- @RequestBody: deserializes the HTTP body (JSON → Java object) — used with POST/PUT/PATCH

@PatchMapping("/customers/{id}/suspend")

public ResponseEntity<?> suspendCustomer(

 @PathVariable UUID id, // from path

 @RequestParam String reason, // from ?reason=...

`@RequestBody SuspendRequest body) { // from JSON body`

Q34. How does @Valid and Bean Validation work?

A: @Valid triggers Hibernate Validator to check JSR-380 annotations on the request object:

- **@NotBlank** — string must not be null or empty
- **@Email** — validates email format with regex
- **@Size(min=8)** — minimum length
- **@Pattern(regex="...")** — custom regex (I used this for password strength)
- **@NotNull** — field required

When validation fails, Spring throws `MethodArgumentNotValidException`. My `GlobalExceptionHandler` catches this and returns a 400 Bad Request with a map of field → error message:

```
{ "password": "Password must contain uppercase, number, and special character" }
```

Q35. How does pagination work with Spring Data JPA?

A: Spring Data JPA's Pageable abstraction is passed to repository methods:

```
Page<Customer> findByStatus(CustomerStatus status, Pageable pageable);
```

Client sends: `GET /customers?page=0&size=20&sort=createdAt,desc`

Spring automatically parses this into a `PageRequest` object. The `Page<T>` response includes:

- **content** — the records for this page
- **totalElements** — total count
- **totalPages** — total page count
- **number** — current page (0-indexed)
- **size** — page size

`@PageableDefault(size = 20)` sets the default page size if client doesn't specify.

The underlying SQL uses `LIMIT 20 OFFSET 0` (or database-specific equivalent).

SECTION 7 — Testing

Q36. What is the difference between unit tests and integration tests? Which did you write?

A:

- Unit tests: Test a single class in isolation. All dependencies are mocked with Mockito. Fast (milliseconds), no Spring context, no DB, no Kafka.
- Integration tests: Test multiple layers together — e.g., Controller + Service + real DB (H2 in-memory). Slower, uses Spring context.

I primarily wrote unit tests for service layer (AuthServiceTest, CustomerServiceTest, BillingServiceTest, etc.) and controller integration tests using @WebMvcTest + MockMvc:

- @WebMvcTest loads only the web layer (no DB, no Kafka)
- MockBean provides mock services
- Tests verify HTTP status, JSON response structure, and that service methods are called

For true integration tests, I'd add @SpringBootTest with @Testcontainers (real PostgreSQL in Docker).

Q37. What is Mockito and how did you use it?

A: Mockito is a mocking framework that creates fake implementations of dependencies:

- @Mock — creates a mock object
- @InjectMocks — creates the class under test, injecting mocks into it
- given(...).willReturn(...) — stubbing: "when this method is called, return this value"
- then(...).should(...) — verification: "assert this method was called"
- willDoNothing().given(...) — stub void methods
- argThat(predicate) — verify method was called with argument matching a condition

Example from my AuthServiceTest:

```
given(userRepository.existsByEmail(request.getEmail())).willReturn(false);

// ... call service ...

then(userRepository).should().save(any(User.class));
```

Q38. What is MockMvc and what did you test with it?

A: MockMvc simulates HTTP requests without starting a real server — it tests the entire web layer (controller, filters, argument resolution, response serialization) without network I/O.

I tested:

- Correct HTTP status codes (201 for register, 200 for login, 400 for bad input)
- Response JSON structure (jsonPath("\$.success").value(true))
- Validation errors (missing/invalid fields return 400)
- Business errors (duplicate email returns 400 with errorCode)

- CSRF token handling (`with(csrf())` for POST requests)
-

SECTION 8 — Observability (ELK, New Relic, Prometheus)

Q39. What is the ELK Stack and what does each component do?

A:

- E — Elasticsearch: A distributed search and analytics engine. Stores log data as JSON documents, indexed for fast full-text and structured queries. In my project, each service's logs go into `index telecom-logs-<service>-<date>`.
- L — Logstash: A data pipeline. Ingests logs from multiple sources (TCP from services, Kafka topics), applies transformations (parsing, field enrichment, tagging errors), and outputs to Elasticsearch.
- K — Kibana: Web UI for Elasticsearch. I use it for: searching logs with KQL, creating dashboards (error rate, consumer lag, billing events), and setting up alerts.

My services use `logstash-logback-encoder` to serialize each log line as JSON and ship it to Logstash via TCP. This means logs arrive structured — no regex parsing needed.

Q40. What is distributed tracing and how does it work?

A: Distributed tracing tracks a single request as it flows through multiple services. Each request gets a unique `traceId`. Each service call within that request creates a `spanId`. Both are propagated via HTTP headers (`traceparent` in W3C format or `X-B3-TraceId` in Zipkin format).

With Micrometer Tracing (Spring Boot 3.2):

- `traceId` and `spanId` are added to Logback's MDC automatically
- All log lines from a request include `[traceId=abc123,spanId=def456]`
- In Kibana, filtering by `traceId: abc123` shows every log line across ALL services for that single user request

New Relic receives spans via the agent and shows a visual waterfall: Gateway → Auth Service → Customer Service, with timing per hop.

Q41. What metrics does your platform expose via Prometheus?

A: With `micrometer-registry-prometheus` and `spring-boot-starter-actuator`, each service exposes `/actuator/prometheus` with:

- `http_server_requests_seconds` — histogram of request latency by URI, method, status
- `jvm_memory_used_bytes` — heap and non-heap memory
- `hikaricp_connections_active` — database connection pool utilization

- `kafka_consumer_records_lag` — messages behind per topic/partition (critical for Usage Service)
- `cache_gets_total{result="hit"/"miss"}` — Redis cache hit rate
- `process_cpu_usage` — CPU load
- `jvm_gc_pause_seconds` — garbage collection pauses

Scraped by Prometheus every 15 seconds, visualized in Grafana dashboards. Alerts fire when e.g. consumer lag > 10,000 or error rate > 5%.

SECTION 9 — Docker & Deployment

Q42. What is a multi-stage Docker build and why did you use it?

A: Multi-stage builds use multiple `FROM` instructions in one Dockerfile. Each stage can copy artifacts from previous stages.

My Dockerfile:

Stage 1: Build (JDK + Maven, ~500MB)

FROM eclipse-temurin:17-jdk-alpine AS builder

RUN ./mvnw package -DskipTests

Result: target/*.jar

Stage 2: Runtime (JRE only, ~180MB)

FROM eclipse-temurin:17-jre-alpine

COPY --from=builder /app/target/*.jar app.jar

Benefits:

- **Smaller final image:** JRE instead of JDK + Maven saves ~400MB per image
- **No source code in production image:** Build tools, source files, and test dependencies aren't shipped
- **Layer caching:** Maven dependencies layer is cached separately from source code layer, so rebuilds that only change source code are fast

Q43. How does Docker Compose help with local development?

A: Docker Compose defines the entire infrastructure as code in one YAML file. With `docker-compose up -d`, it starts:

- 6 PostgreSQL instances (one per service, isolated)

- Redis
- Kafka + Zookeeper + Kafka UI
- A kafka-setup container that creates all 7 topics
- Elasticsearch + Logstash + Kibana
- All 8 microservices (if you've built images)

All services are on the same telecom-network bridge network. Services communicate by container name (e.g., redis, kafka, postgres-auth). Environment variables override application.yml defaults. Health checks ensure dependent services wait for dependencies.

SECTION 10 — Behavioral & Design Questions

Q44. What was the most challenging part of this project?

A (suggested): The most challenging part was designing the event-driven billing pipeline reliably. When a customer signs up, a chain of events must happen: account creation → billing setup → notification. Getting this right with Kafka required:

- Ensuring idempotent consumers (same event processed twice doesn't double-bill)
- Handling partial failures (billing event consumed but notification service is down — message retried from Kafka, not lost)
- Ordering events correctly (service account creation must complete before billing can set up)

Setting up the kafka-setup container to create topics before services start, and configuring `auto.offset.reset=earliest` so services don't miss events published before they start, also required careful testing.

Q45. If you were to scale this platform to 1 million users, what would you change?

A:

1. **Kafka:** Increase partitions for Usage Service topic (currently 6, scale to 30+). Add Kafka cluster with 3 brokers + replication factor 3.
2. **Database:** Add read replicas for Customer/Billing DBs. Consider sharding customer data by region.
3. **Caching:** Redis Cluster (hash slots across 6 nodes) instead of single Redis instance.
4. **API Gateway:** Deploy multiple gateway instances behind a load balancer (NGINX/ALB).
5. **Kubernetes:** Replace Docker Compose with K8s — HPA (Horizontal Pod Autoscaler) based on CPU/Kafka lag metrics. Usage Service autoscales during peak hours.

6. Usage Service: Migrate to Apache Cassandra or TimescaleDB for time-series data instead of partitioned PostgreSQL.
 7. CDN: Cache plan catalogue at edge (CloudFront) — rarely changes.
 8. Observability: Centralize traces in Jaeger/Zipkin, set up PagerDuty alerts from New Relic.
-

Q46. How would you handle a service outage gracefully?

A: Multiple layers:

1. Circuit Breaker (Resilience4j): API Gateway detects repeated failures and opens the circuit. Client gets a 503 with a helpful message instead of a timeout.
 2. Kafka buffering: If Notification Service is down, billing events stay in Kafka (7-day retention). When the service recovers, it catches up — no events lost.
 3. Retry with backoff: RetryConfig in gateway retries failed GET requests 3 times with exponential backoff (50ms → 100ms → 200ms) before triggering circuit breaker.
 4. Health checks: Kubernetes readinessProbe removes unhealthy pods from the load balancer until they recover.
 5. Fallback responses: Gateway's FallbackController returns a JSON response explaining the outage and when to retry — better UX than a raw connection error.
-

Q47. How would you add a new microservice (e.g., Promotions Service)?

A:

1. Create Maven module under services/promotions-service/
2. Add it to root pom.xml modules list
3. Copy application structure: entity, repo, service, controller, DTOs
4. Add DB setup: new PostgreSQL container in docker-compose, Flyway migration
5. Register with Eureka: add spring-cloud-starter-netflix-eureka-client
6. Add route in API Gateway: lb://promotions-service
7. Define Kafka topics if needed (e.g., telecom.promotions.events)
8. Add GatewayJwtFilter for security (same pattern as other services)
9. Add circuit breaker config in gateway
10. Add Prometheus/ELK config, New Relic agent

The modular design makes this straightforward — the common library provides shared DTOs, exception handlers, and events.

Q48. What design patterns did you use in this project?

A:

- **Saga Pattern:** Distributed transactions via choreography (customer creation → Kafka → billing setup → notification)
 - **Circuit Breaker:** Resilience4j in API Gateway prevents cascade failures
 - **API Gateway Pattern:** Single entry point, centralises cross-cutting concerns
 - **Repository Pattern:** Spring Data JPA repositories abstract DB access from service layer
 - **Event-Driven / Publisher-Subscriber:** Kafka topics decouple producers from consumers
 - **DTO Pattern:** Separate request/response objects from entities (MapStruct for mapping)
 - **Factory Pattern:** ProducerFactory, RedisCacheManager builder in config
 - **Builder Pattern:** Lombok @Builder on entities and DTOs for readable object creation
 - **Template Method:** KafkaTemplate abstracts producer details; RedisTemplate abstracts Redis commands
 - **Cache-Aside:** @Cacheable + @CacheEvict for lazy cache population
-

Q49. How would you add rate limiting per user (not per IP)?

A: Spring Cloud Gateway's RequestRateLimiter uses a KeyResolver to identify the "who" being rate-limited. Currently my default key resolver uses the remote IP address. To switch to per-user:

@Bean

```
public KeyResolver userKeyResolver() {  
    return exchange -> {  
        String userId = exchange.getRequest().getHeaders().getFirst("X-User-Id");  
        return Mono.just(userId != null ? userId : "anonymous");  
    };  
}
```

This requires the JWT filter to run first (to inject X-User-Id) before the rate limiter filter — which is handled by filter ordering in Spring Cloud Gateway.

Per-user rate limiting allows legitimate users to have higher quotas than anonymous requests and prevents abuse without blocking all IPs behind a corporate proxy.

Q50. How would you implement audit logging — tracking who changed what?

A: Several approaches:

1. **Kafka Audit Topic:** Every service publishes to `telecom.audit.events` — event includes `userId`, `action`, `entityType`, `entityId`, `oldValue`, `newValue`, `timestamp`. Logstash consumes this and indexes to `telecom-audit-*` in Elasticsearch. Kibana dashboard shows "User X changed customer Y's status from ACTIVE to SUSPENDED at time Z".
2. **JPA Entity Listeners:** `@EntityListeners(AuditListener.class)` on entities — `@PreUpdate/@PrePersist/@PreRemove` hooks capture changes before DB write.
3. **Hibernate Envers:** Automatically versions entity changes in `_AUD` tables — query historical state: `AuditReader.find(Customer.class, customerId, revisionNumber)`.

I currently use approach 1 (Kafka audit topic consumed by Logstash) for simplicity and searchability in Kibana.

End of Interview Q&A Document Total: 50 questions with detailed answers covering Architecture, Security, Kafka, Redis, Spring, JPA, API Design, Testing, Observability, Docker, Behavioral, and Design Patterns.

Real-Time Interview Q&A — TelecomPlatform

"Tell me about your project" style — as if this was your company project

These are the actual questions interviewers ask when you say you've worked on a **microservices-based telecom backend**. Answers are written in first-person so you can speak them naturally.

PART 1 — Project Introduction (Always Asked First)

Q1. Tell me about your current/recent project.

A (speak this naturally):

"I worked on a **telecom backend platform** — think of it like the back-office system behind an ISP like Cox or Comcast. The platform handles everything from customer onboarding, plan subscriptions, monthly billing, real-time usage tracking, to customer support ticketing.

Architecturally, we built it as **8 microservices** using Spring Boot 3.2 and Spring Cloud. Each service owns its own PostgreSQL database. Services talk to each other either synchronously via REST using Feign clients, or asynchronously via **Apache Kafka** for events like invoice generation and notifications.

For security, we used **JWT-based authentication** with refresh token rotation, OAuth2 for social login, and Spring Security for role-based access control. We used **Redis** for caching, rate limiting, and token blacklisting.

On the observability side, we had structured logging flowing into an **ELK Stack**, distributed tracing, and **New Relic APM** for performance monitoring and alerts.

My specific contributions were the Auth Service, the Billing Service, and the event-driven notification pipeline."

Q2. What was your role and what exactly did you build?

A:

"I was a **backend developer** on this project. My primary ownership was:

- **Auth Service** — built the entire authentication module: JWT generation and validation, refresh token rotation, Redis blacklisting on logout, account lockout after failed attempts, and OAuth2 integration with Google and GitHub.
- **Billing Service** — designed the invoice generation engine, payment processing flow, proration logic, and the nightly scheduled job that detects overdue invoices and applies late fees.
- **API Gateway** — set up JWT validation at the gateway level, rate limiting using Redis token bucket, and circuit breakers via Resilience4j.
- **Event pipeline** — designed the Kafka topic architecture and wired up the notification flow so that billing events (invoice generated, payment received) trigger customer emails without the billing service waiting for them.

I also contributed to the shared common library — the `ApiResponse<T>` wrapper, `GlobalExceptionHandler`, and the shared Kafka event POJOs used across services."

Q3. What was the team size and how was work divided?

A:

"We were a team of 5 backend developers, a frontend team of 2, a DevOps engineer, and a product manager. On the backend side, we divided ownership by service:

- I owned Auth and Billing
- Another developer owned Customer Service and Plan Service
- A third handled Usage and Notification
- The fourth worked on Support Service
- Our senior dev oversaw architecture decisions and code reviews

We followed **two-week sprints**, daily standups, and used GitHub pull requests with mandatory review before merge. For API contracts between services, we defined Kafka event schemas and REST API contracts upfront so teams could develop in parallel without blocking each other."

Q4. What tech stack did you use and why those specific choices?

A:

Spring Boot 3.2 — it's the industry standard for enterprise Java backends. The auto-configuration and Spring ecosystem meant we didn't reinvent wheels for security, caching, or messaging.

Kafka over RabbitMQ — we chose Kafka because of message retention. Kafka keeps messages for 7 days, so if the Notification Service went down, it would catch up on missed billing events when it recovered. With RabbitMQ, those messages would be gone. Also, Kafka's consumer group model lets multiple services consume the same event stream independently.

PostgreSQL over MySQL — better support for advanced features we needed: table partitioning for the Usage Service's high-volume data, UUID primary keys natively, and better JSON support.

Redis — we needed it in three places: caching for read-heavy endpoints like plan catalogue and customer lookups, rate limiting at the API gateway, and token blacklisting on logout. Redis's atomic operations via Lua scripts made the rate limiting reliable.

New Relic over Datadog — purely a cost decision at the company level. Both are great APM tools."

PART 2 — Challenges (The Most Important Section)

Q5. What was the most challenging part of this project?

A:

"Honestly, the most challenging part was **getting the billing event pipeline right under failure conditions**.

The flow is: when a customer makes a payment, Billing Service processes it, marks the invoice PAID, publishes a PAYMENT_RECEIVED Kafka event, and the Notification Service consumes it to send a payment confirmation email.

The problem we hit in testing was a **race condition**: the Kafka event was sometimes published *before* the database transaction committed. So when the Notification Service received the event, called back to the Billing Service to get invoice details, and the invoice still showed as PENDING in the DB — the email said 'Your payment is being processed' instead of 'Payment confirmed.'

We fixed this by using **Transactional Outbox Pattern** conceptually — we made sure the Kafka publish happened *after* the transaction committed using Spring's `@TransactionalEventListener(phase = AFTER_COMMIT)`. The event only goes to Kafka after the DB row is fully committed. That completely eliminated the race condition.

It was a subtle bug but a very important one for billing accuracy — you never want a payment confirmation email to be inconsistent with the actual payment status."

Q6. Did you face any performance issues? How did you resolve them?

A:

"Yes — two significant ones.

First one was in the Usage Service. Network equipment pushes usage events to Kafka, and we needed to persist them to PostgreSQL at high volume — bursts of 500+ events per second during peak hours. Initially, we were inserting records one-by-one via JPA's `save()`, which was creating individual INSERT statements. At peak load, the DB write lag started growing.

We solved this by:

1. Setting `hibernate.jdbc.batch_size=50` and `hibernate.order_inserts=true` so Hibernate batches 50 inserts into one DB round-trip
2. Increasing `max.poll.records=500` in the Kafka consumer so we process 500 events in one poll cycle
3. Moving to PostgreSQL table partitioning so the `usage_records` table is split by month — queries only scan the relevant month's partition instead of the full table

After these changes, peak write throughput improved by roughly 8x.

Second issue was N+1 queries in the Customer Service. When listing customers with their service accounts for the CSR dashboard, Hibernate was firing one query for the customer list and then one query *per customer* to load their accounts. With 50 customers on a page, that's 51 queries.

We fixed it with a JOIN FETCH in the JPQL query:

```
@Query("\nSELECT c FROM Customer c LEFT JOIN FETCH c.serviceAccounts WHERE c.status = :status\n")
```

That brought 51 queries down to 1."

Q7. Tell me about a bug that was hard to find and fix.

A:

"There was a bug in our **JWT token blacklist** that only appeared under load testing.

When a user logged out, we stored their token in Redis with a TTL. The bug was: under high concurrency, multiple logout requests for the same token (from different browser tabs) would sometimes both pass the 'is this token blacklisted' check simultaneously — because the Redis check and the blacklist SET were two separate operations. A tiny window existed where both requests read 'not blacklisted' before either wrote to Redis.

It wasn't a security hole because we also revoked the refresh token in PostgreSQL. But it did cause the logout endpoint to return success twice without error, which confused our frontend.

We fixed it by replacing the two-step check-then-set with a Redis SET NX (set if not exists) — a single atomic operation. If the key already exists, the SET fails silently. This is the same pattern used for distributed locks.

```
Boolean result = redisTemplate.opsForValue()
```

```
.setIfAbsent(BLACKLIST_PREFIX + token, \"revoked\", ttl, TimeUnit.MILLISECONDS);
```

Finding it required **load testing with 50 concurrent logout requests** — it never appeared in normal testing. That taught me to always test race conditions explicitly, not assume they won't happen."

Q8. How did you handle a situation where a downstream service was down?

A:

"This actually happened during our integration testing phase. The Notification Service went down, and we were worried billing events would be lost.

What saved us was **Kafka's message retention**. The billing events stayed in the telecom.billing.events topic — Kafka doesn't care if consumers are down. When Notification Service came back online, it resumed from where it left off using its committed offset. All the invoice emails were delivered a few minutes late, but nothing was lost.

However, we did have a problem with the **circuit breaker on the API Gateway side**. When the Notification Service was down, the gateway's health check was showing it as unhealthy, but the circuit breaker wasn't triggering because the service was completely unreachable, not returning 5xx errors — the TCP connection was just timing out. The default timeoutDuration was 60 seconds, so the gateway was hanging for a full minute on each request routed to that service.

We fixed this by reducing timeoutDuration to 10 seconds in the Resilience4j config and also adding a TimeLimiter so slow responses were treated as failures for circuit breaker calculation. After that, the circuit opened within 3 failed attempts."

Q9. How did you handle data consistency between microservices?

A:

"This is one of the hardest problems in microservices, and honestly we made pragmatic trade-offs.

We accepted **eventual consistency** for most cross-service data. For example, when a customer is suspended, the Customer Service updates the status in its DB and publishes a `CUSTOMER_STATUS_CHANGED` Kafka event. The Billing Service consumes this and puts a hold on future invoice generation. There's a window of maybe a few hundred milliseconds where Customer Service shows the customer as `SUSPENDED` but Billing Service hasn't processed the event yet.

For our use case — ISP account management — this was acceptable. It's not a banking system where consistency must be instant.

For **critical operations like payments**, we used a different approach. The entire payment flow — validate invoice, insert payment record, update invoice status, and publish event — all happened within a single database transaction on the Billing Service. The Kafka event only got published after successful commit. So within one service, we had ACID guarantees. Across services, we had eventual consistency.

The one place we enforced stronger consistency was invoice payment deduplication. The `transactionId` column has a `UNIQUE` constraint, so even if the same payment event was processed twice, the second DB insert would fail and we'd handle it gracefully."

Q10. Tell me about a design decision you made that you'd change in hindsight.

A:

"I'd change how we handled the **refresh token storage**.

We stored refresh tokens in the Auth Service's PostgreSQL database. As users logged in from multiple devices — phone, laptop, tablet — each got its own refresh token row. After a few months in production simulation, that table would have millions of rows. We had a scheduled cleanup job to delete expired tokens, but querying for active tokens per user required an indexed lookup on `user_id` and `revoked=false`.

What I'd do differently is store refresh tokens **only in Redis** with a `refreshToken:<userId>:<deviceId>` key pattern, using Redis's native TTL for automatic expiry. No cleanup job needed, $O(1)$ lookup by key, and Redis is already part of our infrastructure.

The trade-off is that Redis is in-memory — if Redis goes down and you haven't set up persistence (AOF/RDB), all refresh tokens are invalidated and users are logged out. But with Redis persistence configured and a replica, that's an acceptable risk for stateless token storage.

I learned that just because you *can* put something in PostgreSQL doesn't mean you *should*. Ephemeral data with TTL-based expiry is a natural fit for Redis."

PART 3 — Technical Deep Dives (What Interviewers Drill Into)

Q11. How exactly does your token rotation work? Walk me through the code flow.

A:

"When a client calls `/api/v1/auth/refresh-token` with their refresh token:

1. We look up the token in the `refresh_tokens` table
2. Check two conditions: `revoked = false AND expires_at > NOW()` — if either fails, we throw a 401
3. If valid, we **immediately mark the old token as revoked** in the same transaction — `refreshToken.setRevoked(true); repo.save(refreshToken);`
4. Generate a brand new access token (15 min) and a new refresh token (7 days)
5. Persist the new refresh token to the DB
6. Return both new tokens to the client

The key security benefit of rotation: if an attacker steals a refresh token and tries to use it *after* the legitimate user has already refreshed, the stolen token is already marked revoked — they get a 401. And if the attacker uses it *first*, the legitimate user's next refresh attempt gets a 401, which is a signal to the system that token theft may have occurred — you could trigger account security alerts at this point."

Q12. You mentioned Redis for rate limiting. What happens if Redis is down?

A:

"Good question — this is a failure scenario we explicitly handled.

Spring Cloud Gateway's `RequestRateLimiter` filter has a `denyEmptyKey` and `status` behavior config. By default, if the Redis call to check rate limit fails — connection timeout, Redis unreachable — the filter throws an exception and the request gets a 500 Internal Server Error, which is terrible UX.

We configured a fallback: if the Redis rate limit check fails, we **allow the request through** (fail-open) rather than blocking all traffic. The reasoning is: rate limiting is a protection mechanism, not a core business requirement. If Redis is down, we'd rather serve requests and deal with potential abuse than take down the entire platform.

`spring.cloud.gateway.default-filters:`

- name: RequestRateLimiter

args:

`deny-empty-key: false # don't reject if key resolution fails`

`empty-key-status-code: 200 # allow through on failure`

For a security-critical API like authentication endpoints, we'd fail-closed. For general API traffic, fail-open is the right call."

Q13. How does your Billing Service handle a scenario where the same invoice is paid twice?

A:

"Two layers of protection.

First layer — application level: Before processing a payment, we check `invoice.getStatus() == PAID`. If true, we throw a `BusinessException('Invoice already paid')` and return a 400 Bad Request. No payment gateway call is made.

Second layer — database level: The payments table has a `transaction_id` column with a `UNIQUE` constraint. Even if two concurrent payment requests somehow both passed the application check simultaneously (race condition), only one `INSERT` would succeed. The second would throw a `DataIntegrityViolationException`, which we catch and translate to a 409 Conflict response.

The `transactionId` is generated as `TXN- + UUID` before the payment gateway call. In a real system with an actual Stripe/Braintree integration, the payment gateway returns its own transaction ID which you'd store — making deduplication even more robust because the gateway itself deduplicates."

Q14. Your Usage Service ingests events from Kafka. What happens if an event has corrupted/invalid data?

A:

"This was a real problem we handled — network equipment can send malformed payloads.

In the Kafka consumer, we wrapped the entire processing logic in a try-catch:

```
try {  
    UsageRecord record = mapToRecord(event); // can throw if fields missing/invalid  
    usageRepository.save(record);  
    ack.acknowledge(); // only ACK on success  
} catch (Exception e) {  
    log.error("Failed to process usage event: {}", event, e);  
    // do NOT call ack.acknowledge()  
}
```

If we don't ACK, Kafka will redeliver the message. But we don't want to keep retrying a fundamentally bad message forever — that would block processing of valid messages behind it (in the same partition).

We configured a `DeadLetterPublishingRecoverer` via Spring Kafka's `SeekToCurrentErrorHandler` with max 3 retries and backoff:

@Bean

```
public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> template) {  
    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(template);  
    return new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 3));  
}
```

}

After 3 attempts, the bad message goes to telecom.usage.events.DLT. We have a Kibana dashboard alert on DLT message count. The DevOps team investigates, fixes the equipment sending bad data, and can replay the DLT messages after fixing the root cause."

Q15. How did you manage secrets like database passwords and JWT keys in your project?

A:

"In local development, we used Docker Compose environment variables — defined in a .env file that's in .gitignore, so it never gets committed.

In production-like environments, we used **environment variable injection** at container startup — the actual secrets come from the deployment system (Kubernetes Secrets, AWS Secrets Manager, or HashiCorp Vault) and are injected as environment variables that Spring Boot reads via `${DB_PASSWORD}` placeholders in application.yml.

The JWT secret is a 256-bit random key encoded in Base64 — long enough to make brute-force infeasible. We never hardcode it:

jwt:

secret: \${JWT_SECRET} # injected at runtime, never in source code

One thing I'd improve: use **Spring Cloud Config Server** or **AWS Parameter Store** for centralized config management, with dynamic refresh so you can rotate secrets without restarting services. Currently, a JWT secret rotation requires a rolling restart of all services."

Q16. How did you handle schema migrations in a running system without downtime?

A:

"This is where Flyway's incremental migration model really shines.

Each migration is a new numbered SQL file — we never modify existing migrations, only add new ones. For example, if we needed to add a `credit_score` column to the customers table:

- `V2__add_credit_score_to_customers.sql`:
- `ALTER TABLE customers ADD COLUMN credit_score INT;`

Flyway applies this on service startup. For zero-downtime deploys:

1. We deploy the **DB migration first** as a separate step — the new column exists but no code uses it yet
2. Then deploy the **new application code** that starts reading/writing the column
3. If rollback is needed, the column stays (with NULLs) — the old code ignores it

This **expand-contract pattern** ensures the DB and app can be deployed independently. We never do breaking schema changes in a single step — that would require coordinated downtime.

For the partitioned `usage_records` table, we add new monthly partition files in a separate migration file each month:

```
-- V5__add_usage_partition_2024_07.sql

CREATE TABLE usage_records_2024_07 PARTITION OF usage_records
    FOR VALUES FROM ('2024-07-01') TO ('2024-08-01');

--
```

****Q17. How did your services communicate internally? Did you use Feign clients?****

****A.****

"We used Feign clients for ****synchronous service-to-service calls**** — specifically where the Billing Service needed to verify a customer exists before generating an invoice, or where the Support Service needed to look up account details.

```
```java

@FeignClient(name = \"customer-service\", fallback = CustomerServiceFallback.class)

public interface CustomerServiceClient {

 @GetMapping(\"/api/v1/customers/{id}\")

 CustomerResponse getCustomer(@PathVariable UUID id);

}
```

The `name` attribute matches the Eureka service name — no hardcoded URLs. Spring Cloud LoadBalancer picks the appropriate instance.

The fallback class handles cases where Customer Service is down — returns a cached version or a sensible default to prevent the Billing Service from failing entirely.

For most cross-service communication though, we preferred **Kafka events** because they decouple the services temporally — Billing doesn't need Customer Service to be up to generate an invoice if it already has the customer data it needs. We stored `customerEmail` and `accountNumber` denormalized in the Billing Service DB for exactly this reason — avoid runtime cross-service calls on the critical billing path."

---

**Q18. Explain a situation where you had to optimize a slow API endpoint.**

**A:**

"The /api/v1/customers/search endpoint was running at 800ms–1.2 seconds, which was unacceptable for the CSR dashboard that CSRs use all day.

I profiled it with New Relic and found two problems:

**Problem 1 — Missing database index.** The search query filtered by status and tier columns but had no composite index on them. PostgreSQL was doing a full sequential scan on 100k+ customer rows. Fix:

```
CREATE INDEX idx_customers_status_tier ON customers(status, customer_tier);
```

This alone dropped the query from 600ms to 80ms.

**Problem 2 — The search was doing LIKE queries without taking advantage of indexes.**

LOWER(email) LIKE '%query%' with a leading wildcard can't use a B-tree index. For a name/email search, PostgreSQL needs a full-text index or trigram index.

```
CREATE EXTENSION pg_trgm;
```

```
CREATE INDEX idx_customers_email_trgm ON customers USING GIN (email gin_trgm_ops);
```

After both fixes, the endpoint went from 800ms to under 100ms. Then I added Redis caching for the most common search filters (e.g., status=ACTIVE, tier=GOLD) with a 5-minute TTL, which brought repeated searches to under 10ms.

New Relic's transaction trace was essential here — it showed exactly which SQL query was the bottleneck with the actual execution time."



## PART 4 — Team & Process Questions

---

### Q19. How did you handle API versioning when you needed to change an endpoint?

**A:**

"We had this exact situation when we changed the CustomerResponse DTO to add nested addresses in the response — the old response had a flat streetLine1, city etc. at the root.

We couldn't just change it because the frontend was already consuming the old format. Our approach:

1. **Keep the old endpoint working** at /api/v1/customers/{id}
2. Deploy the new format at /api/v2/customers/{id} alongside it
3. Frontend team migrated their calls to v2 over the next sprint
4. After 2 weeks with zero v1 calls (confirmed from New Relic request counts), we deprecated v1 with a Deprecation response header warning
5. Removed v1 after another sprint

The lesson learned: **define a deprecation policy early**. We should have planned for versioning from day one rather than retrofitting it. Going forward, we added a rule: any breaking API change requires a new version endpoint, with a minimum 4-week deprecation window."

---

**Q20. Did you have any conflicts with teammates about technical decisions? How did you handle it?**

**A:**

"Yes — there was a disagreement about whether to use **Kafka or REST callbacks** for the notification flow.

A teammate argued for REST callbacks — when billing generates an invoice, it directly POSTs to the Notification Service's API. Simpler to debug, immediate delivery, and fewer moving parts.

My argument was for Kafka — decoupled, resilient to Notification Service downtime, and no tight dependency between Billing and Notification.

We resolved it by listing the failure modes for each approach:

- With REST callback: if Notification Service is down, billing invoice generation fails or we need complex retry logic in the Billing Service. Billing now has to know about Notification — coupling we don't want.
- With Kafka: Billing doesn't care if Notification is down. Messages are retained. Notification catches up.

Once we framed it around failure resilience rather than personal preference, the team aligned on Kafka. The key was making the decision objective — not 'I prefer Kafka' but 'Kafka handles this failure mode that REST callbacks don't.'"

---

**Q21. How did you do code reviews? What did you look for?**

**A:**

"We had mandatory 1-reviewer PRs for regular features and 2-reviewer for anything touching security, payments, or database schema.

What I personally looked for when reviewing:

- **Security:** Any SQL built from string concatenation (injection risk), hardcoded secrets, sensitive data logged, authentication bypasses
- **Transactions:** Methods that do multiple DB operations — are they wrapped in `@Transactional`? What's the propagation level?
- **Null safety:** Any `NullPointerException` risks — especially with Optionals not being handled
- **Exception handling:** Are exceptions being swallowed silently? Are business exceptions giving enough context?
- **Test coverage:** Does the PR include tests? Do the tests actually test failure paths, not just the happy path?

- **Kafka consumers:** Is the `ack.acknowledge()` call only happening after successful processing? Not at the start of the method?
- **Redis:** Are all cache evictions in place when data is updated? Did they set appropriate TTLs?

The things that got most PRs bounced back were: missing `@Transactional` on multi-step operations, missing test for the negative/failure case, and exceptions being caught and logged but not re-thrown when they should propagate."

---

## Q22. How did you debug issues in production (or staging)? Walk me through your process.

A:

"Our debugging workflow had a clear order:

**Step 1 — New Relic first.** Check the error rate dashboard. New Relic shows which endpoint is spiking, the stack trace of the exception, and the exact SQL or external call that failed. This narrows it to a service and a method in under a minute.

**Step 2 — Kibana logs.** Take the `traceId` from New Relic, paste it into Kibana's KQL: `traceId: \"abc123\"`. This shows every log line across ALL services for that one request — in chronological order. You see exactly where in the chain things broke.

**Step 3 — Kafka UI.** If the issue looks like an event processing problem, check the Kafka UI at port 9093 — look at consumer lag for the relevant group, check if any messages are in the DLT topic.

**Step 4 — Database.** If needed, connect to the specific service's PostgreSQL and query directly to verify data state.

Real example: A customer reported they received an invoice email but couldn't find the invoice in the portal. In Kibana, filtering by their `customerId` showed Billing Service had published the Kafka event *successfully*, but the Notification Service log showed it consumed the event and called back to Billing API — which returned a 404 because the invoice ID in the event had a typo (UUID truncated to 35 chars instead of 36 due to a formatting bug). Fixed the UUID serialization in the event producer, and the problem never recurred."

---

## Q23. How did you ensure the system was reliable? What was your testing strategy?

A:

"We had three layers:

**Unit tests** — every service layer class had unit tests covering happy path and at least 2–3 failure cases. We used JUnit 5 + Mockito. Goal was 80% line coverage on service classes.

**Integration tests** — the auth flow and billing payment flow had end-to-end tests using `@SpringBootTest` with `TestContainers` (real PostgreSQL and Redis in Docker). These ran on every PR merge to main.

**Manual testing checklist** — for anything involving Kafka or multi-service flows, we had a testing checklist: start all services locally with Docker Compose, run through key flows manually, verify Kafka events appear in Kafka UI, verify emails triggered in Notification Service logs.

What we *didn't* have that I wish we did: **contract testing with Pact**. When the Customer Service changed their API response format, the Billing Service's Feign client broke silently — we only found out when an integration test failed. Pact would have caught that at the unit test level without needing a full environment."

---

## 🔥 PART 5 — Scenario-Based (Interviewers Love These)

---

**Q24. Imagine the database connection pool is exhausted. What would happen and how would you diagnose it?**

**A:**

"With HikariCP (Spring Boot's default pool), if all connections are in use and a new request needs one, the request waits up to `connectionTimeout` (30 seconds in our config). After that, HikariCP throws `SQLTransientConnectionException: Connection is not available, request timed out after 30000ms`.

Symptoms: API response times spike to ~30 seconds, then 500 errors. New Relic would show `hikaricp_connections_pending` metric climbing and `hikaricp_connections_active` pegged at `maximumPoolSize`.

Diagnosis steps:

1. New Relic → JVM metrics → HikariCP dashboard — confirm pool exhaustion
2. PostgreSQL query: `SELECT * FROM pg_stat_activity WHERE state = 'active' ORDER BY duration DESC` — find long-running or blocked queries
3. Check application logs for queries that are taking unusually long — likely a missing index causing a full table scan and holding a connection open

Root causes I've seen:

- A new query without an index, causing full table scans taking 10+ seconds per request
- A deadlock — two transactions waiting for each other's locks
- `@Transactional` method doing external HTTP calls (Feign) while holding a DB connection open — the external call is slow, connection is tied up waiting

Fix for the external call scenario: move the Feign call *before* the `@Transactional` method, pass the result in, so the DB connection is only held during the actual DB operations."

---

**Q25. A customer says they were charged twice. How would you investigate?**

**A:**

"This is a P1 — I'd work through it systematically.

**Step 1 — Find the customer's invoices.** Query the invoices table by `customer_id`, look for duplicate invoice numbers or two invoices with the same billing period.

**Step 2 — Check the payments table.** Look for two COMPLETED payment records for the same invoice. Check transaction\_id — if both have different transaction IDs, the payment gateway actually processed two payments. If same transaction ID, something went wrong with our deduplication.

**Step 3 — Check Kafka events.** In Kafka UI, look at telecom.billing.events — did we publish two PAYMENT\_RECEIVED events for the same invoice? If yes, trace back to what triggered the duplicate publish.

**Step 4 — Check application logs.** Using the invoice ID, pull all log lines from Kibana. Look for duplicate API calls to /api/v1/payments — maybe the frontend retried a payment request that was slow.

**Most likely root cause in our system:** The frontend clicked 'Pay' twice (or a timeout caused a retry), sending two POST requests to /api/v1/payments. Our application-level guard checks invoice.status == PAID — but if both requests arrived within milliseconds of each other, both might have passed the check before either committed to DB.

Our fix was the DB-level UNIQUE constraint on transaction\_id. But if the frontend sent two requests with *different* transaction IDs generated client-side, both would succeed.

**Long-term fix:** Implement idempotency keys — the client sends a unique Idempotency-Key header. The server stores processed idempotency keys in Redis with a short TTL. Duplicate requests with the same key return the original response without reprocessing.

**Immediate action for the customer:** Refund the duplicate charge via the payment gateway's refund API and send a courtesy email."

---

## Q26. Your Notification Service is sending duplicate emails. How do you fix it?

A:

"This actually happened once in staging. The Notification Service was restarted mid-processing, and because we were using auto-commit offset mode at the time, some events had their offsets committed before the email was actually sent. On restart, those events weren't reprocessed. That wasn't the duplicate issue — that was actually the *loss* issue.

The duplicate issue was different: **consumer rebalancing**. When the Notification Service scaled from 1 to 2 instances, Kafka rebalanced partitions. During rebalancing, one partition was briefly assigned to both the old and new consumer. With manual commit, both consumers processed the same batch before one surrendered the partition.

Fixes we applied:

1. **Manual ACK** — commit offset only after email is successfully sent, not before
2. **Idempotency on the consumer side** — before sending an email, check Redis for a key notification:sent:<eventId>. If the key exists, skip sending and just ACK. If not, send email and then set the key in Redis with a 24-hour TTL.

```
String eventId = (String) event.get("eventId");
```

```
Boolean alreadySent = redisTemplate.opsForValue()
```

```

 .setIfAbsent("\"notification:sent:\" + eventId, \"sent\", 24, HOURS);
 if (Boolean.FALSE.equals(alreadySent)) {
 log.warn(\"Duplicate event {} - skipping\", eventId);
 ack.acknowledge();
 return;
 }
 // send email
 ack.acknowledge();

```

Every Kafka event in our system includes a UUID eventId generated at publish time specifically to enable this pattern."

---

**Q27. How would you add a feature where premium customers (GOLD/PLATINUM) get a 10% discount on their invoice?**

**A:**

"I'd implement this in the Billing Service's invoice generation logic.

First, the Billing Service needs to know the customer's tier. Since Billing and Customer are separate services, I'd either:

- Call the Customer Service via Feign client to get tier at invoice generation time, OR
- Have the Customer Service include tier in the SERVICE\_ACCOUNT\_ADDED Kafka event, and Billing stores it locally (denormalization)

I prefer the second — no runtime dependency on Customer Service during billing.

In the BillingService.generateInvoice():

```

double discountRate = 0.0;
if (\"GOLD\".equals(request.getCustomerTier())) {
 discountRate = 0.05; // 5% for Gold
} else if (\"PLATINUM\".equals(request.getCustomerTier())) {
 discountRate = 0.10; // 10% for Platinum
}

```

```

double discountAmount = Math.round(subtotal * discountRate * 100.0) / 100.0;

```

I'd add a line item for the discount:

- Connect Plus Monthly Service: \$49.99

- Regulatory Recovery Fee:     \$5.99  
- PLATINUM Loyalty Discount (10%): -\$4.99  
- Tax (8%):                     \$4.08  
Total:                         \$55.07

Then add a DB migration:

```
ALTER TABLE invoices ADD COLUMN discount_reason VARCHAR(100);
```

Tests I'd add:

- Gold customer gets 5% discount
- Platinum customer gets 10% discount
- Standard customer gets 0% discount
- Discount is applied to subtotal before tax calculation (not after)
- Discount appears as a line item on the invoice"

---

## Q28. The team wants to add SMS notifications. How would you extend the system?

A:

"The Notification Service is already designed as an event consumer — adding SMS is an extension, not a redesign.

**Step 1 — Add SMS provider SDK.** We'd use Twilio or AWS SNS. Add the dependency to notification-service's pom.xml.

**Step 2 — Create SmsService:**

@Service

```
public class SmsService {

 private final TwilioClient twilioClient;

 public void sendSms(String toPhoneNumber, String message) {
 Message.creator(
 new PhoneNumber(toPhoneNumber),
 new PhoneNumber(TWILIO_FROM_NUMBER),
 message
).create();
 }
}
```



**Step 3 — Customer preference.** We already have preferredContactMethod on the Customer entity (EMAIL, PHONE, SMS, MAIL). The Notification Service would call the Customer Service API or read from a Kafka customer event to know the customer's preference.

**Step 4 — Update the event consumers:**

```
case \"INVOICE_GENERATED\" -> {
 if (\"SMS\".equals(customer.getPreferredContactMethod())) {
 smsService.sendSms(customer.getPhoneNumber(),
 \"Your TelecomPlatform invoice #\" + invoiceNumber + \" for $\" + amount + \" is due on \" +
 dueDate);
 } else {
 notificationService.sendInvoiceNotification(event); // email
 }
}
```

**Step 5 — Rate limiting for SMS.** Unlike email, SMS costs money per message. Add a Redis counter to prevent accidentally spamming a customer: sms:count:<customerId>:<date> — cap at 5 SMS per customer per day.

No changes needed to any other service — Kafka decoupling means the billing, customer, and support services don't know or care how notifications are delivered."

---

## PART 6 — "Why" Questions (Tests Your Depth)

---

**Q29. Why did you use @Builder from Lombok everywhere? What's the benefit?**

**A:**

"Three reasons:

1. **Readability** —  
Customer.builder().firstName(\"John\").email(\"j@x.com\").status(ACTIVE).build() vs a constructor with 15 parameters where you lose track of which position is which.
2. **Immutable-friendly** — You construct the full object at once rather than creating it with a no-arg constructor and then calling 10 setters. Fewer states an object can be in means fewer bugs.
3. **Test data setup** — In tests,  
Customer.builder().id(UUID.randomUUID()).email(\"test@x.com\").build() is clean and self-documenting. You only set the fields relevant to each test, leaving others at defaults.

The one issue with @Builder: it doesn't set @Builder.Default values when you use it on an entity that also has @NoArgsConstructor (required by JPA). JPA needs a no-arg constructor, but Lombok's

@Builder doesn't call the field initializers when building. You have to explicitly put @Builder.Default on fields that need non-null defaults:

@Builder.Default

```
private List<ServiceAccount> serviceAccounts = new ArrayList<>();
```

Without this, customer.getServiceAccounts() would throw NPE when accessing the list via the builder-constructed object."

---

### Q30. Why did you choose MapStruct over manual mapping or ModelMapper?

A:

"Three options we considered:

**Manual mapping** — writing response.setFirstName(entity.getFirstName()) for every field. Zero magic, perfect control, but 50 lines of boilerplate for a complex entity. Maintenance nightmare when you add fields.

**ModelMapper** — reflection-based, maps by field name convention at runtime. Easy to use but: reflection is slow (matters at scale), it's a black box (hard to debug mismatches), and it fails silently when field names don't match.

**MapStruct** — generates the mapping code at compile time as plain Java. No reflection, so it's as fast as manual mapping. It fails at compile time if a field is unmapped (rather than silently at runtime). You can see exactly what code it generates in target/generated-sources.

```
@Mapper(componentModel = "spring")
```

```
public interface CustomerMapper {
```

```
 @Mapping(target = "id", expression = "java(customer.getId().toString())")
```

```
 CustomerResponse toResponse(Customer customer);
```

```
}
```

For us, the compile-time safety was the deciding factor. When we renamed a field in the Customer entity, MapStruct immediately gave a compile error pointing to every mapper that used the old field name — we fixed them all before any runtime failure."

---

*End of Practical Interview Q&A Total: 30 real-world questions — framed as company project experience*

---

### Quick-Reference: Phrases to Use in Interviews

Use these naturally to sound experienced:

- "We made a pragmatic trade-off between **consistency and availability**..."
- "The failure scenario we were most worried about was..."

- "In hindsight I'd change..."
- "New Relic's transaction trace showed us exactly where..."
- "The bug only appeared under concurrent load, which is why unit tests didn't catch it..."
- "We followed the expand-contract pattern for schema migrations to avoid downtime..."
- "The Kafka consumer group rebalancing caused..."
- "We used idempotency keys to handle duplicate requests..."
- "I profiled it and found the bottleneck was N+1 queries / missing index / pool exhaustion..."
- "We resolved the disagreement by framing it around failure scenarios rather than preferences..."